

Unidad VII: Grafos

Clase 12: Modelar relaciones complejas: Grafos

Objetivo

Comprender cómo modelar relaciones no jerárquicas.

Tema

- Definición de grafo.
- Matriz de adyacencia.
- Lista de adyacencia.
- Especificación del TDA.

Grafos: modelado de relaciones complejas

El estudio de grafos permite representar problemas donde las relaciones entre elementos no siguen una estructura jerárquica. En un árbol existe una organización padre-hijo, con una raíz y caminos descendentes. En un grafo, en cambio, los elementos pueden conectarse de múltiples maneras: una ciudad puede tener vuelos hacia varias ciudades, un usuario puede relacionarse con muchos usuarios, un sistema puede depender de varios servicios, una cuenta bancaria puede estar vinculada con distintas operaciones y una red logística puede tener múltiples rutas alternativas.

Esta estructura resulta especialmente útil cuando interesa modelar conexiones, caminos, dependencias, circulación, alcance o comunicación entre entidades. Por eso los grafos aparecen en sistemas de transporte, redes de computadoras, plataformas sociales, comercio electrónico, mapas, auditoría de procesos, motores de recomendación y análisis de fraude. La idea central no es ordenar los elementos en niveles, sino representar cómo se vinculan entre sí.

El apunte de cátedra ubica los grafos dentro del estudio de estructuras complejas y los trabaja desde tres dimensiones: conceptos básicos, especificación del Tipo de Dato Abstracto y posibles implementaciones mediante matriz de adyacencia o lista de adyacencia.

Concepto de grafo

Un grafo es una estructura formada por vértices y aristas. Los vértices, también llamados nodos, representan los elementos del problema. Las aristas representan las relaciones o conexiones entre esos elementos.

En una red de vuelos, los vértices pueden ser ciudades y las aristas pueden ser vuelos directos. En una red social, los vértices pueden ser usuarios y las aristas pueden representar amistades, seguimientos o interacciones. En un sistema de logística, los

vértices pueden ser depósitos, sucursales o centros de distribución, mientras que las aristas pueden representar rutas disponibles entre ellos.

Formalmente, un grafo suele representarse como:

$$G = (V, A)$$

donde V es el conjunto de vértices y A es el conjunto de aristas. Si el grafo representa una red de vuelos entre Buenos Aires, Córdoba, Mendoza y Bariloche, el conjunto de vértices podría ser:

$$V = \{\text{Buenos Aires, Córdoba, Mendoza, Bariloche}\}$$

y el conjunto de aristas podría ser:

$$A = \{(\text{Buenos Aires, Córdoba}), (\text{Córdoba, Mendoza}), (\text{Mendoza, Bariloche})\}$$

Cada par indica que existe una conexión directa entre dos vértices. La forma de interpretar esa conexión depende del problema. En un mapa de rutas puede representar un camino físico; en una red bancaria puede representar una transferencia; en una plataforma digital puede representar una interacción entre usuarios.

Vértices, aristas y dirección de la relación

La primera decisión importante al modelar un problema con grafos consiste en determinar si las relaciones tienen dirección. Esta diferencia define si el grafo será dirigido o no dirigido.

Un grafo no dirigido representa relaciones simétricas. Si existe una arista entre A y B , se interpreta que la conexión vale en ambos sentidos. Por ejemplo, si dos computadoras están conectadas por un enlace físico bidireccional, puede representarse con una arista no dirigida. Lo mismo ocurre con una amistad recíproca en una red social: si Ana es amiga de Bruno, Bruno también es amigo de Ana.

Un grafo dirigido representa relaciones con sentido. En este caso, una arista de A hacia B no implica automáticamente una arista de B hacia A . En una red de vuelos, puede existir vuelo directo desde Buenos Aires hacia Mendoza, pero no necesariamente desde Mendoza hacia Buenos Aires. En una red social tipo “seguidores”, una persona puede seguir a otra sin que esa relación sea recíproca. En auditoría de procesos, una tarea puede depender de otra, pero la dependencia no funciona en sentido inverso.

La notación (A, B) suele leerse como una arista que va desde A hacia B . El primer vértice es el origen y el segundo es el destino. En un grafo dirigido, (A, B) y (B, A) son relaciones distintas.

Grafos ponderados

Un grafo es ponderado cuando sus aristas tienen un valor asociado, llamado peso. Ese peso representa una medida relevante para el problema: distancia, costo, tiempo, capacidad, riesgo, prioridad, cantidad de operaciones o cualquier valor que permita comparar conexiones.

En una red de vuelos, el peso puede representar el costo del pasaje o la duración estimada del viaje. En logística, puede representar kilómetros recorridos, tiempo de entrega o costo de transporte. En una red de sistemas, puede representar latencia o carga de comunicación entre servicios.

Por ejemplo:

(Buenos Aires, Córdoba, 700)
(Córdoba, Mendoza, 650)
(Mendoza, Bariloche, 1200)

Cada arista conserva el origen, el destino y el peso. Esto permite resolver preguntas más ricas que la simple existencia de una conexión. No solo interesa saber si hay vuelo directo entre dos ciudades, sino cuánto cuesta, cuánto tarda o qué ruta conviene seleccionar.

En los materiales de referencia se trabaja con grafos dirigidos, sin aristas paralelas ni bucles, con nodos enteros y pesos enteros positivos, lo cual simplifica el análisis inicial y permite concentrarse en las operaciones fundamentales de la estructura.

Aristas paralelas, bucles y nodos aislados

Existen algunos conceptos que conviene diferenciar desde el inicio porque suelen confundirse.

Una arista paralela aparece cuando hay más de una arista con el mismo origen y el mismo destino. Por ejemplo, si se registran dos vuelos distintos desde Buenos Aires hacia Córdoba como aristas separadas, habría aristas paralelas. En algunos modelos esto puede tener sentido, por ejemplo cuando cada vuelo tiene horario, aerolínea y precio propio. En una introducción al TDA Grafo suele evitarse esta complejidad, representando una única conexión entre cada par de vértices.

Un bucle es una arista cuyo origen y destino son el mismo vértice. Por ejemplo, (A, A). En algunos problemas puede representar una operación que vuelve al mismo estado, pero en modelos básicos de grafos suele excluirse para simplificar el comportamiento.

Un nodo aislado es un vértice que no tiene aristas entrantes ni salientes. Puede existir dentro del grafo aunque no esté conectado con otros vértices. En una red de vuelos, una ciudad cargada en el sistema pero sin vuelos disponibles sería un nodo aislado. En una red social, un usuario registrado sin vínculos con otros usuarios también puede modelarse de ese modo.

Adyacencia, grado, camino y alcanzabilidad

Dos vértices son adyacentes cuando existe una arista entre ellos. En un grafo dirigido, la adyacencia debe interpretarse con cuidado, porque la relación tiene sentido. Si existe una arista desde A hacia B, entonces B es adyacente desde A como destino alcanzable directamente. Sin embargo, eso no significa que A sea alcanzable directamente desde B.

El grado de un vértice describe cuántas conexiones tiene. En grafos dirigidos se distingue entre grado positivo y grado negativo. El grado positivo indica la cantidad de aristas salientes de un vértice. El grado negativo indica la cantidad de aristas entrantes. Por ejemplo, si Buenos Aires tiene vuelos hacia Córdoba, Mendoza e Iguazú, su grado positivo es 3. Si recibe vuelos desde Bariloche y Salta, su grado negativo es 2. El apunte de cátedra utiliza esta distinción para analizar la cantidad de aristas entrantes y salientes de un nodo dentro de un grafo dirigido.

Un camino es una sucesión ordenada de vértices conectados por aristas. Por ejemplo:

Buenos Aires → Córdoba → Mendoza → Bariloche

Este camino indica que no necesariamente existe un vuelo directo desde Buenos Aires hacia Bariloche, pero sí una forma de llegar mediante conexiones intermedias. En un grafo ponderado, el costo del camino se calcula sumando los pesos de las aristas que lo componen. Si Buenos Aires → Córdoba cuesta 700, Córdoba → Mendoza cuesta 650 y Mendoza → Bariloche cuesta 1200, el costo total del camino es 2550.

Un vértice es alcanzable desde otro si existe al menos un camino que permite llegar desde el origen hasta el destino. La alcanzabilidad es uno de los conceptos más importantes en grafos porque permite responder preguntas como: “¿se puede llegar desde esta ciudad hasta otra?”, “¿un proceso puede alcanzar cierto estado?”, “¿un usuario está conectado indirectamente con otro?” o “¿un sistema puede comunicarse con un servicio determinado?”.

Un ciclo es un camino que comienza y termina en el mismo vértice. Por ejemplo:

Buenos Aires → Córdoba → Mendoza → Buenos Aires

Los ciclos requieren especial atención porque pueden generar recorridos infinitos si no se controla qué vértices ya fueron visitados. Este punto es una diferencia esencial entre recorrer árboles y recorrer grafos: en un grafo puede haber vueltas, caminos alternativos y conexiones circulares. El apunte remarca que los ciclos deben tratarse cuidadosamente al recorrer grafos para evitar quedar atrapados en un camino sin salida lógico.

Ejemplo aplicado: red de vuelos

Una red de vuelos regionales puede modelarse mediante un grafo dirigido y ponderado. Las ciudades serán vértices, los vuelos directos serán aristas dirigidas y el costo estimado de cada vuelo será el peso de la arista.

Se consideran las siguientes ciudades:

- 1 = Buenos Aires
- 2 = Córdoba
- 3 = Mendoza
- 4 = Bariloche
- 5 = Iguazú
- 6 = Salta

Y los siguientes vuelos directos:

Buenos Aires → Córdoba, costo 700
 Buenos Aires → Mendoza, costo 980
 Buenos Aires → Iguazú, costo 1100
 Córdoba → Mendoza, costo 650
 Mendoza → Bariloche, costo 1200
 Bariloche → Buenos Aires, costo 1600
 Iguazú → Salta, costo 900
 Salta → Córdoba, costo 600

Este modelo permite observar varias propiedades. Buenos Aires tiene tres vuelos salientes. Córdoba recibe vuelos desde Buenos Aires y Salta, pero solo tiene un vuelo saliente hacia Mendoza. Bariloche tiene una salida hacia Buenos Aires. Además, existe un camino desde Iguazú hasta Bariloche aunque no haya vuelo directo: Iguazú → Salta → Córdoba → Mendoza → Bariloche.

La red no tiene forma de árbol. No existe una raíz única, no hay una jerarquía padre-hijo y pueden existir ciclos. Por ejemplo, Buenos Aires puede llegar a Córdoba, Córdoba a Mendoza, Mendoza a Bariloche y Bariloche nuevamente a Buenos Aires. Esa relación circular es natural en redes reales de transporte.

Matriz de adyacencia

La matriz de adyacencia representa un grafo mediante una matriz cuadrada. Las filas representan vértices de origen y las columnas representan vértices de destino. Si existe una arista desde el vértice de la fila hacia el vértice de la columna, se registra un valor en la celda correspondiente. Si no existe arista, se registra 0, NULL u otro valor definido por convención.

En un grafo no ponderado, suele colocarse 1 cuando existe arista y 0 cuando no existe. En un grafo ponderado, en lugar de colocar 1 se coloca el peso de la arista. El material MADO indica que en grafos ponderados puede colocarse el peso de la arista (i, j) en la

posición correspondiente de la matriz, y utilizar 0 o NULL para indicar ausencia de conexión.

Para la red de vuelos, la matriz de adyacencia ponderada puede representarse así:

Origen \ Destino	BUE	COR	MDZ	BRC	IGR	SLA
BUE	0	700	980	0	1100	0
COR	0	0	650	0	0	0
MDZ	0	0	0	1200	0	0
BRC	1600	0	0	0	0	0
IGR	0	0	0	0	0	900
SLA	0	600	0	0	0	0

La celda ubicada en la fila Buenos Aires y columna Córdoba contiene 700, por lo tanto existe un vuelo directo desde Buenos Aires hacia Córdoba con costo 700. La celda ubicada en la fila Córdoba y columna Buenos Aires contiene 0, por lo tanto no existe vuelo directo desde Córdoba hacia Buenos Aires en el modelo planteado.

Esta representación permite responder muy rápido si existe una conexión directa entre dos vértices. Para saber si hay vuelo directo desde Mendoza hacia Bariloche, basta con consultar la fila Mendoza y la columna Bariloche. Esta consulta tiene un acceso directo sobre la matriz.

La principal desventaja de la matriz es el uso de memoria. Para n vértices se necesita una matriz de $n \times n$, aunque haya pocas aristas. Si una red tiene 1000 ciudades, la matriz tendría 1.000.000 de posiciones. Si solo existen 3000 vuelos directos, la mayoría de las celdas estarían vacías o en cero. Por eso la matriz de adyacencia suele convenir cuando el grafo es denso, es decir, cuando existen muchas aristas en relación con la cantidad máxima posible, o cuando la consulta frecuente consiste en verificar rápidamente si existe una conexión entre dos vértices.

Lista de adyacencia

La lista de adyacencia representa un grafo asociando a cada vértice la lista de sus vértices adyacentes. En un grafo ponderado, cada elemento de la lista puede guardar también el peso de la arista.

Para la red de vuelos, la lista de adyacencia puede expresarse así:

BUE $\rightarrow$ (COR, 700), (MDZ, 980), (IGR, 1100)
 COR $\rightarrow$ (MDZ, 650)
 MDZ $\rightarrow$ (BRC, 1200)
 BRC $\rightarrow$ (BUE, 1600)
 IGR $\rightarrow$ (SLA, 900)
 SLA $\rightarrow$ (COR, 600)

Esta representación es más compacta cuando el grafo tiene muchas entidades pero pocas conexiones reales. Solo se almacenan las aristas existentes. Si una ciudad no tiene vuelo directo hacia otra, simplemente no aparece en su lista.

El material MADO define esta representación como un arreglo de listas, donde cada posición corresponde a un vértice y cada lista contiene los vértices alcanzables mediante aristas salientes. En grafos dirigidos, la suma de los largos de las listas coincide con la cantidad de aristas; en grafos no dirigidos, cada arista aparece dos veces, una por cada sentido.

La lista de adyacencia es especialmente adecuada cuando se necesita responder preguntas como: “¿cuáles son los destinos directos desde esta ciudad?”, “¿qué usuarios están conectados con este usuario?”, “¿qué tareas dependen de esta tarea?” o “¿qué sistemas reciben información desde este servicio?”. También es útil para recorridos, porque desde cada vértice se accede naturalmente a sus vecinos directos.

Su desventaja aparece cuando la consulta principal es saber si existe una arista específica entre dos vértices. En una matriz, esa pregunta se responde consultando una celda. En una lista, puede ser necesario recorrer la lista de adyacentes del vértice origen hasta encontrar el destino o confirmar que no existe.

Criterios para elegir una representación

La elección entre matriz de adyacencia y lista de adyacencia no es solo una decisión de programación; es una decisión de diseño. Depende del tipo de grafo, del volumen de datos y de las consultas más frecuentes.

La matriz de adyacencia conviene cuando el grafo es pequeño o denso, cuando muchas combinaciones de vértices están conectadas, y cuando la operación más frecuente es verificar si existe una conexión directa entre dos vértices. También es una representación didácticamente clara, porque permite visualizar todas las relaciones posibles en una estructura tabular.

La lista de adyacencia conviene cuando el grafo es disperso, es decir, cuando hay muchos vértices pero pocas aristas. También es preferible cuando interesa recorrer vecinos, listar conexiones salientes o ejecutar algoritmos de exploración. En sistemas reales, muchas redes son dispersas: no todas las ciudades tienen vuelos directos a todas las demás, no todos los usuarios interactúan con todos, no todos los depósitos se conectan con todos los puntos de entrega.

Un error frecuente consiste en elegir matriz de adyacencia solo porque parece más simple de entender. Esa simplicidad puede ser costosa si el grafo crece mucho y la mayoría de las posiciones quedan vacías. Otro error común consiste en usar lista de adyacencia sin considerar que algunas consultas directas pueden requerir búsqueda dentro de la lista. La representación correcta depende de qué se necesita optimizar: memoria, consulta directa, recorrido o flexibilidad.

Tipo de Dato Abstracto Grafo

Un Tipo de Dato Abstracto, o TDA, define qué operaciones ofrece una estructura y qué comportamiento se espera de ellas, sin imponer una única forma de implementación. En grafos, el TDA permite separar el concepto lógico de grafo de su representación concreta. El mismo TDA puede implementarse con matriz de adyacencia, con lista de adyacencia o con otras estructuras más complejas.

La especificación básica del TDA Grafo incluye operaciones para inicializar la estructura, agregar y eliminar vértices, agregar y eliminar aristas, consultar si existe una arista, obtener el peso de una arista y recuperar el conjunto de vértices. El apunte de cátedra presenta estas operaciones como base de la especificación GrafoTDA.

Una especificación posible en Java es:

```
public interface GrafoTDA {

    void InicializarGrafo();

    // Agrega un vértice al grafo.
    // Precondición: el grafo está inicializado y el vértice no existe.
    void AgregarVertice(int v);

    // Elimina un vértice del grafo.
    // Precondición: el grafo está inicializado y el vértice existe.
    void EliminarVertice(int v);

    // Devuelve el conjunto de vértices del grafo.
    // Precondición: el grafo está inicializado.
    ConjuntoTDA Vertices();

    // Agrega una arista desde v1 hacia v2 con un peso.
    // Precondición: existen ambos vértices y no existe previamente la arista.
    void AgregarArista(int v1, int v2, int peso);

    // Elimina la arista desde v1 hacia v2.
    // Precondición: existe la arista.
    void EliminarArista(int v1, int v2);

    // Indica si existe una arista desde v1 hacia v2.
    // Precondición: existen ambos vértices.
    boolean ExisteArista(int v1, int v2);

    // Devuelve el peso de la arista desde v1 hacia v2.
    // Precondición: existe la arista.
    int PesoArista(int v1, int v2);
}
```

Esta interfaz no dice si el grafo se guarda en una matriz, en una lista o en otra estructura. Solo define el contrato de comportamiento. Cualquier implementación concreta debe respetar esas operaciones.

Comportamientos básicos del grafo

Las operaciones del TDA pueden agruparse según su finalidad. Esta clasificación ayuda a comprender qué tipo de comportamiento ofrece la estructura.

Las operaciones de construcción permiten crear el grafo y cargar sus componentes. InicializarGrafo prepara la estructura para ser usada. AgregarVertice incorpora un nuevo nodo. AgregarArista crea una conexión entre dos vértices existentes.

Las operaciones de modificación permiten cambiar el estado del grafo. EliminarVertice quita un nodo y, según la implementación, también debe resolver qué ocurre con las aristas asociadas a ese nodo. EliminarArista elimina una conexión puntual entre dos vértices.

Las operaciones de consulta permiten obtener información sin alterar la estructura. Vertices devuelve el conjunto de nodos cargados. ExisteArista indica si hay una conexión directa entre dos vértices. PesoArista recupera el valor asociado a una conexión existente.

Sobre estas operaciones básicas pueden construirse comportamientos derivados. Por ejemplo, obtener todos los adyacentes de un vértice, calcular el grado positivo, saber si existe un camino entre dos vértices, determinar si un vértice es alcanzable desde otro o recorrer el grafo. Estos comportamientos no siempre forman parte de la especificación mínima del TDA, pero se apoyan en sus operaciones.

En la red de vuelos, ExisteArista(BUE, MDZ) permitiría saber si hay vuelo directo desde Buenos Aires hacia Mendoza. PesoArista(BUE, MDZ) devolvería el costo de ese vuelo. Para saber si Buenos Aires puede llegar a Bariloche con una o más escalas, ya no alcanza con consultar una sola arista: hace falta recorrer el grafo.

Implementación mediante matriz de adyacencia

Una implementación estática puede usar una matriz de adyacencia. En esta estrategia se reserva una matriz donde cada celda representa la posible existencia de una arista entre dos vértices.

El apunte de cátedra describe la matriz como una estructura cuadrada en la que el contenido de cada celda representa el peso de la arista cuyo origen está dado por la fila y cuyo destino está dado por la columna. También señala un problema importante: en Java, los índices de una matriz son consecutivos y comienzan en cero, mientras que los vértices del grafo pueden tener valores no continuos. Por eso se requiere un mecanismo de mapeo entre el valor real del vértice y su posición interna en la matriz.

Por ejemplo, si los vértices son {10, 25, 80}, no se puede usar directamente la posición 80 de una matriz pequeña. Se necesita una estructura auxiliar que indique que el vértice 10 está en la posición 0, el vértice 25 en la posición 1 y el vértice 80 en la posición 2.

Una estructura simplificada podría incluir:

```
int[][] matrizAdyacencia;
int[] etiquetas;
int cantidadVertices;
```

La matriz guarda los pesos de las aristas. El arreglo etiquetas guarda el valor real de cada vértice. La variable cantidadVertices indica cuántos vértices fueron cargados. Cuando se desea agregar o consultar una arista, primero se busca qué índice interno corresponde a cada etiqueta de vértice.

Esta implementación permite que ExisteArista(v1, v2) sea eficiente una vez encontrados los índices internos, porque alcanza con revisar la celda correspondiente. Sin embargo, agregar y eliminar vértices puede requerir cuidado adicional, especialmente si se busca mantener compacto el uso de la matriz.

Implementación mediante lista de adyacencia

Una implementación dinámica puede usar listas de adyacencia. En esta estrategia se mantiene una lista de vértices, y cada vértice conserva una lista de sus aristas salientes. Cada arista apunta al vértice destino y guarda el peso de la conexión.

El apunte de cátedra presenta esta idea mediante dos estructuras: NodoGrafo, que representa cada vértice, y NodoArista, que representa cada conexión saliente desde un nodo. Cada nodo de grafo contiene su valor, una referencia al siguiente nodo y una referencia a su lista de aristas. Cada nodo de arista contiene el peso, una referencia al nodo destino y una referencia a la siguiente arista.

Una representación conceptual sería:

```
class NodoGrafo {
    int nodo;
    NodoArista arista;
    NodoGrafo sigNodo;
}

class NodoArista {
    int peso;
    NodoGrafo nodoDestino;
    NodoArista sigArista;
}
```

En esta implementación, agregar una arista desde v1 hacia v2 requiere encontrar el nodo origen y el nodo destino. Luego se inserta una nueva arista en la lista de adyacencia del nodo origen. La estructura resulta flexible y adecuada para grafos donde la cantidad de conexiones reales es mucho menor que la cantidad de conexiones posibles.

Para la red de vuelos, el nodo Buenos Aires tendría una lista de aristas hacia Córdoba, Mendoza e Iguazú. El nodo Córdoba tendría una arista hacia Mendoza. El nodo Bariloche tendría una arista hacia Buenos Aires. Esta forma de almacenamiento se parece mucho a cómo se piensa el problema: para cada ciudad, se guardan sus destinos directos.

Recorridos en grafos

Recorrer un grafo significa visitar sus vértices siguiendo sus aristas. A diferencia de un árbol, un grafo no tiene necesariamente una raíz única ni una estructura jerárquica. Además, puede contener ciclos. Por eso todo recorrido de grafos necesita dos decisiones básicas: desde qué vértice comenzar y cómo registrar qué vértices ya fueron visitados.

El registro de visitados evita procesar varias veces el mismo vértice y previene recorridos infinitos cuando existen ciclos. En algunos algoritmos se usan marcas simples, como visitado o no visitado. En otros se utilizan estados más detallados: blanco para vértices no descubiertos, gris para vértices descubiertos pero no terminados, y negro para vértices completamente procesados. El material MADO utiliza esta lógica de colores para explicar recorridos como BFS y DFS.

Los dos recorridos clásicos son BFS y DFS.

Recorrido en anchura

BFS significa Breadth-First Search, traducido como búsqueda o recorrido en anchura. Su estrategia consiste en visitar primero los vecinos directos del vértice inicial, luego los vecinos de esos vecinos, y así sucesivamente. Avanza por niveles de distancia respecto del origen.

Para implementar esta lógica se utiliza una cola. La cola permite procesar primero los vértices que fueron descubiertos antes, respetando el comportamiento FIFO: First In, First Out, es decir, primero en entrar, primero en salir.

En una red de vuelos, si el recorrido comienza en Buenos Aires, primero se visitarían sus destinos directos: Córdoba, Mendoza e Iguazú. Luego se visitarían los destinos directos de esas ciudades que todavía no hayan sido visitados. Desde Córdoba se podría llegar a Mendoza, pero si Mendoza ya fue descubierta, no se repite. Desde Iguazú se llegaría a Salta. Desde Mendoza se llegaría a Bariloche.

La utilidad de BFS es muy clara cuando interesa conocer qué vértices son alcanzables desde un origen y a qué “distancia” en cantidad de pasos se encuentran. En una red social, permite encontrar contactos a uno, dos o tres grados de separación. En una red de vuelos, permite identificar ciudades alcanzables con una escala, dos escalas o más. En un sistema de soporte, puede servir para explorar dependencias cercanas antes de avanzar hacia dependencias más lejanas.

Recorrido en profundidad

DFS significa Depth-First Search, traducido como búsqueda o recorrido en profundidad. Su estrategia consiste en avanzar por un camino todo lo posible antes de retroceder y explorar alternativas pendientes. Puede implementarse mediante recursividad o

mediante una pila. La pila responde al comportamiento LIFO: Last In, First Out, es decir, último en entrar, primero en salir.

El material MADO describe DFS como una exploración sistemática de aristas donde primero se visitan los vértices adyacentes al vértice más recientemente descubierto. Cuando ya no quedan adyacentes sin visitar, el algoritmo retrocede, proceso conocido como backtracking, para continuar con otros caminos pendientes.

En la red de vuelos, partiendo desde Buenos Aires, DFS podría avanzar a Córdoba, luego a Mendoza, luego a Bariloche, y desde Bariloche volver a Buenos Aires. Como Buenos Aires ya estaría visitado, no se repite. Luego el recorrido retrocedería para explorar otros destinos pendientes de Buenos Aires, como Iguazú y Salta.

DFS es útil cuando interesa explorar caminos completos, detectar ciclos, analizar dependencias profundas o recorrer componentes de una red. En sistemas de software, puede aplicarse para analizar cadenas de dependencias entre módulos. En procesos de auditoría, puede servir para seguir una ruta completa de eventos o controles relacionados.

Diferencia entre BFS y DFS

BFS y DFS recorren grafos, pero responden a estrategias distintas. BFS explora primero lo cercano; DFS explora primero lo profundo. BFS usa cola; DFS usa pila o recursividad. BFS es natural cuando importa la cantidad mínima de pasos en grafos no ponderados. DFS es natural cuando interesa explorar caminos completos o analizar estructuras de dependencia.

En una red de vuelos, BFS sería adecuado para responder: “¿qué ciudades puedo alcanzar con la menor cantidad de escalas?”. DFS sería adecuado para explorar rutas posibles completas desde una ciudad de origen.

Ambos recorridos necesitan controlar vértices visitados. Sin ese control, un ciclo puede provocar repeticiones indefinidas. Esta es una de las diferencias más importantes respecto de los recorridos de árboles, donde la estructura jerárquica evita naturalmente ciertos tipos de ciclos.

Operaciones básicas y algoritmos derivados

Es importante distinguir entre operaciones básicas del TDA y algoritmos derivados sobre el grafo. Las operaciones básicas son parte del contrato mínimo de la estructura: agregar vértices, agregar aristas, eliminar elementos y consultar conexiones. Los recorridos, en cambio, son algoritmos que utilizan esas operaciones para resolver problemas de exploración.

Por ejemplo, `ExisteArista(A, B)` responde si hay conexión directa entre A y B. Pero “¿B es alcanzable desde A?” es una pregunta más amplia: puede existir un camino

indirecto. Para responderla, se necesita recorrer el grafo desde A y verificar si B aparece durante la exploración.

Del mismo modo, `PesoArista(A, B)` devuelve el peso de una conexión directa. Pero “¿cuál es el camino más barato entre A y B?” requiere un algoritmo adicional. Esa pregunta ya no pertenece a la especificación mínima del TDA, sino al uso algorítmico del grafo.

Esta separación evita confusiones. El TDA define las capacidades estructurales básicas. Los algoritmos usan esas capacidades para resolver problemas más complejos.

Errores frecuentes al modelar grafos

Un error frecuente es confundir grafo con árbol. Si el problema tiene múltiples conexiones posibles, ciclos, caminos alternativos o relaciones entre pares de elementos sin jerarquía, probablemente no convenga forzarlo como árbol. Una red de vuelos no tiene raíz natural; una ciudad puede conectarse con muchas otras y pueden existir rutas circulares.

Otro error común es ignorar la dirección de las aristas. Si se modela como no dirigido un problema donde la relación tiene sentido, se generan resultados incorrectos. Un vuelo de Buenos Aires a Córdoba no garantiza el vuelo de Córdoba a Buenos Aires. Una transferencia de una cuenta a otra no implica una transferencia inversa.

También es frecuente omitir el peso cuando el valor de la conexión es relevante. Si todas las aristas se tratan igual, se pierde información clave para decidir rutas, costos, prioridades o riesgos. En logística, no es lo mismo una ruta de 10 kilómetros que una de 500. En auditoría, no es lo mismo una conexión de bajo riesgo que una de alto riesgo.

Finalmente, debe evitarse elegir una representación sin considerar el tipo de consulta. Una matriz puede ser simple, pero consumir demasiada memoria. Una lista puede ahorrar espacio, pero requerir más búsqueda para verificar una conexión puntual. El diseño debe responder al uso esperado de la estructura.

Síntesis conceptual

Un grafo permite modelar relaciones complejas entre entidades. Sus elementos principales son vértices y aristas. Cuando las aristas tienen sentido, el grafo es dirigido. Cuando tienen un valor asociado, el grafo es ponderado. Conceptos como adyacencia, grado, camino, ciclo y alcanzabilidad permiten analizar cómo se conectan los elementos y qué posibilidades de recorrido existen dentro de la red.

La matriz de adyacencia y la lista de adyacencia son dos formas de representar la misma estructura. La matriz favorece la consulta directa entre pares de vértices, pero puede desperdiciar memoria en grafos dispersos. La lista de adyacencia almacena solo

las conexiones existentes y resulta más natural para recorrer vecinos, aunque puede requerir más búsqueda para comprobar una arista puntual.

El TDA Grafo define las operaciones esenciales de la estructura sin imponer una implementación única. Esa separación permite comprender que el comportamiento abstracto del grafo puede mantenerse estable aunque cambie su representación interna. Sobre esa base se construyen algoritmos de recorrido como BFS y DFS, que permiten explorar vértices alcanzables, analizar caminos y trabajar con redes donde pueden existir ciclos, conexiones múltiples y rutas alternativas.

Especificación del TDA

Para utilizar un grafo, definiremos las siguientes operaciones:

- InicializarGrafo: inicializa el grafo.
- AgregarVertice: agrega un nuevo vértice al grafo, siempre que el grafo esté inicializado y el vértice no exista.
- EliminarVertice: elimina el vértice dado siempre que el vértice exista.
- vértices: devuelve el conjunto de vértices de un grafo, siempre que el grafo esté inicializado.
- AgregarArista: agrega una arista al grafo entre los vértices dados y con el peso dado, siempre que existan ambos vértices pero no exista una arista entre ambos.
- EliminarArista: elimina la arista entre los vértices dados, siempre que esta arista exista.
- ExisteArista: indica si el grafo contiene una arista entre los vértices dados, siempre que los vértices existan.
- PesoArista: devuelve el peso de la arista entre los vértices dados, siempre que la arista exista.

Entonces queda la siguiente especificación del GrafoTDA:

```
public interface GrafoTDA { void
    InicializarGrafo();
    // siempre que el grafo esté inicializado y no exista el nodo
    void AgregarVertice(int v);
    // siempre que el grafo esté inicializado y exista el nodo
    void EliminarVertice(int v);
    // siempre que el grafo esté inicializado
    ConjuntoTDA Vertices();
    // siempre que el grafo esté inicializado, no exista la arista, pero existan ambos nodos
    void AgregarArista(int v1, int v2, int peso);
    // siempre que el grafo esté inicializado y exista la arista
    void EliminarArista(int v1, int v2);
    // siempre que el grafo esté inicializado y existan los nodos
    boolean ExisteArista(int v1, int v2);
    // siempre que el grafo esté inicializado y exista la arista
    int PesoArista(int v1, int v2);
}
```

Implementaciones estáticas y dinámicas

Vamos a proponer dos implementaciones del TDA, una estática y otra dinámica. La implementación estática es una implementación con matrices de adyacencia. Una matriz de adyacencia es una matriz cuadrada donde la cantidad de filas y columnas está dada por la cantidad de nodos del grafo. Además el contenido de cada celda es el peso de la arista con origen en el número de fila y destino en el número de columna.

Ahora bien, en Java, los índices de las matrices van de cero en adelante, en forma consecutiva. No es el caso de los nodos, que pueden tener valores no continuos. Por lo tanto, vamos a tener que utilizar un mapeo que nos indique en qué índice de la matriz está el nodo que buscamos. Entonces, al nombre del nodo lo vamos a llamar internamente etiqueta.

Por ejemplo, para el grafo dado de la figura 6.3, la matriz sería la que se ve en la figura 6.4 y el mapeo entre etiquetas e índices se puede ver en la figura 6.5

	0	1	2	3	4	5	6	7	8
0	0	12	21	0	0	0	0	0	0
1	10	0	0	0	0	0	0	0	0
2	0	0	0	32	9	0	0	0	0
3	0	0	0	0	0	10	0	0	0
4	0	0	0	0	0	12	0	0	0
5	0	0	0	0	87	0	0	0	0
6	0	0	0	0	0	0	0	0	0
7	0	0	0	0	0	0	0	0	10
8	0	0	0	0	0	0	0	0	0

Figura 6.4: Matriz de adyacencia

1	2	3	4	5	6	7	8	10
0	1	2	3	4	5	6	7	8

Etiquetas de mapeo entre vértices e índices de la matriz

```
public class GrafoMA implements GrafoTDA {

    static int n = 100;
    int [][] MAdy;
    int [] Etiqs;
    int cantNodos;
    public void InicializarGrafo() {
```

```

MAdy = new int[n][n];
Etiqs = new int[n];
cantNodos = 0;
}

public void AgregarVertice( int v) {
    Etiqs[cantNodos] = v;
    for ( int i=0; i<=cantNodos; i++){
        MAdy[cantNodos][i] = 0;
        MAdy[i][ cantNodos] = 0;
    }
    cantNodos++;
}

public void EliminarVertice( int v) {
    int ind = Vert2Indice(v);
    for ( int k=0; k<cantNodos; k++)
        MAdy[k][ind] = MAdy [k][cantNodos -1];
    for ( int k=0; k<cantNodos; k++)
        MAdy[ind][k] = MAdy [cantNodos -1][ k];
    Etiqs[ind] = Etiqs[cantNodos -1];
    cantNodos --;
}

private int Vert2Indice( int v){
    int i=cantNodos -1;
    while(i>=0 && Etiqs[i]!=v)
        i--;
    return i;
}

public ConjuntoTDA Vertices() {
    ConjuntoTDA Vert =new ConjuntoLD();
    Vert .InicializarConjunto();
    for ( int i=0; i<cantNodos; i++){
        Vert.Agregar(Etiqs[i]);
    }
    return Vert ;
}

public void AgregarArista( int v1 , int v2 , int peso) {
    int o = Vert2Indice(v1);
    int d = Vert2Indice(v2);
    MAdy [o][d] = peso ;
}

public void EliminarArista( int v1 , int v2) {
    int o = Vert2Indice(v1);
    int d = Vert2Indice(v2);
    MAdy [o][d]=0;
}

public boolean ExisteArista( int v1 , int v2) {
    int o = Vert2Indice(v1);
    int d = Vert2Indice(v2);
    return MAdy [o][d]!=0;
}

```

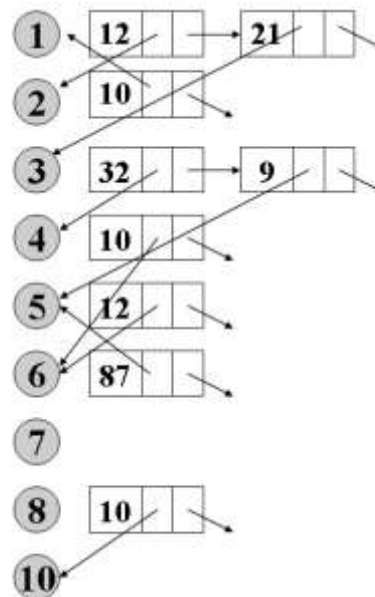
```

public int PesoArista( int v1 , int v2) {
    int o = Vert2Indice(v1);
    int d = Vert2Indice(v2);
    return MAdy [o][d];
}

```

Para el caso de la implementación dinámica, llamaremos a la implementación lista de adyacencia. La idea es tener una lista encadenada de nodos (a los que llamaremos **NodoGrafo**). Cada nodo tiene su valor, una referencia al próximo nodo y además una referencia a la lista de aristas que tienen a ese nodo como origen. Una arista (a la que llamaremos **NodoArista**), tiene una referencia al nodo destino de esta arista, el peso y una referencia a la próxima arista de esa lista.

Entonces para el grafo del ejemplo tenemos la la representación donde los círculos son los **NodoGrafo** y los rectángulos los **NodoArista**.



Lista de adyacencia

La representación de los nodos y las aristas es la siguiente:

```

class NodoGrafo{
    int nodo;
    NodoArista arista;
    NodoGrafo sigNodo;
}

```

```

class NodoArista{
    int etiqueta;
    NodoGrafo nodoDestino;
    NodoArista sigArista;
}

```

Entonces, la implementación con listas de adyacencia es la siguiente:

```
public class GrafoLA implements GrafoTDA{

    NodoGrafo origen;

    public void InicializarGrafo(){
        origen = null;
    }

    public void AgregarVertice(int v){
        //El vertice se inserta al inicio de la lista de nodos
        NodoGrafo aux = new NodoGrafo();
        aux.nodo = v;
        aux.arista = null;
        aux.sigNodo = origen;
        origen = aux;
    }

    /*
     * Para agregar una nueva arista al grafo, primero se deben
     * buscar los nodos entre los cuales se va agregar la arista,
     * y luego se inserta sobre la lista de adyacentes del nodo
     * origen (en este caso nombrado como v1)
     */
    public void AgregarArista(int v1, int v2, int peso){
        NodoGrafo n1 = Vert2Nodo(v1);
        NodoGrafo n2 = Vert2Nodo(v2);

        //La nueva arista se inserta al inicio de la lista
        //de nodos adyacentes del nodo origen
        NodoArista aux = new NodoArista();
        aux.etiqueta = peso;
        aux.nodoDestino = n2;
        aux.sigArista = n1.arista;
        n1.arista = aux;
    }

    private NodoGrafo Vert2Nodo(int v){
        NodoGrafo aux = origen;
        while (aux!=null && aux.nodo!=v){
            aux = aux.sigNodo;
        }
        return aux;
    }
}
```

```
Public void EliminarVertice( int v){
    //Se recorre la lista de v´ertices para remover el nodo v
    //y las aristas con este v´ertice.
    //Distingue el caso que sea el primer nodo
    if (origen.nodo == v) {
```

```

        origen = origen.sigNodo;
    }
    NodoGrafo aux = origen;
    while (aux != null) {
        //remueve de aux todas las aristas hacia v
        this.EliminarAristaNodo(aux, v);
        if (aux.sigNodo != null && aux.sigNodo.nodo == v) {
            //Si el siguiente nodo de aux es v, lo elimina
            aux.sigNodo = aux.sigNodo.sigNodo;
        }
        aux = aux.sigNodo;
    }
}
/*
 * Si en las aristas del nodo existe
 * una arista hacia v, la elimina
 */
private void EliminarAristaNodo(NodoGrafo nodo, int v){
    NodoArista aux = nodo.arista;
    if (aux != null) {
        //Si la arista a eliminar es la primera en
        //la lista de nodos adyacentes
        if (aux.nodoDestino.nodo == v){
            nodo.arista = aux.sigArista;
        }
        else
        {
            while (aux.sigArista != null && aux.sigArista.nodoDestino.nodo != v){
                aux = aux.sigArista;
            }
            if (aux.sigArista != null) {
                //Quita la referencia a la arista hacia v
                aux.sigArista = aux.sigArista.sigArista;
            }
        }
    }
}
}

public ConjuntoTDA Vertices(){
    ConjuntoTDA c = new ConjuntoLD();

```

```

        NodoGrafo aux = origen;
        while (aux != null){
            c.Agregar(aux.nodo);
            aux = aux.sigNodo;
        }
        return c;
    }

    /*
     * Se elimina la arista que tiene como origen al vértice v1
     * y destino al vértice v2
     */
    public void EliminarArista(int v1, int v2){
        NodoGrafo n1 = Vert2Nodo(v1);
        EliminarAristaNodo(n1,v2);
    }

    public boolean ExisteArista(int v1, int v2){
        NodoGrafo n1 = Vert2Nodo(v1);

        NodoArista aux = n1.arista;
        while (aux!=null && aux.nodoDestino.nodo!=v2){
            aux = aux.sigArista;
        }
        //Solo si se encontro la arista buscada, aux no es null
        return aux!=null;
    }

    public int PesoArista(int v1, int v2){
        NodoGrafo n1 = Vert2Nodo(v1);

        NodoArista aux = n1.arista;
        while (aux.nodoDestino.nodo != v2){
            aux = aux.sigArista;
        }
        //Se encontró la arista entre los dos nodos
        return aux.etiqueta;
    }
}

```